

END-TO-END PROJECT MANUAL

Brand Visibility Intelligence Dashboard

*Step-by-step implementation, complete source code, validation and
operations*

Version 1.0 | 21 August 2026 | Python · SQLite · Streamlit · Plotly

How to use this guide

This manual is both a learning path and a technical reference. Follow Chapters 1-12 in order for a first implementation. Use the appendices when copying or reviewing the complete project source.

Recommended workflow: Create the environment, run the demo pipeline, verify the dashboard, then replace demo input with the supplied dirty CSV and finally enable SerpAPI.

Table of contents

In Microsoft Word: right-click the table above and select Update Field to refresh page numbers.

1. Project objective and deliverables

The system measures how products and brands perform in shopping search results. It combines external API results with an existing CSV, cleans and enriches the records, stores them in SQLite, answers 30 analytical questions and serves an interactive dashboard.

Deliverable	Implementation
Source code	Installable Python package under src/brand_visibility
Clean dataset	data/processed/cleaned_products.csv
SQL serving layer	data/database/brand_visibility.db
Dashboard	Six-tab Streamlit application with dynamic SQL filters
EDA report	reports/eda_answers.md with all 30 questions
Quality controls	Tests, lint configuration, Docker health check and cleaning report

2. Architecture and folder structure

```

Brand/
├── config/           # Logging and runtime configuration
├── data/{raw,interim,processed,database}/
├── docs/             # Architecture and this manual
├── reports/          # EDA, cleaning diagnostics and figures
├── scripts/          # Operational scripts
├── src/brand_visibility/ # Installable application package
│   ├── analytics/    # EDA and business insight logic
│   ├── dashboard/    # Streamlit presentation layer
│   ├── data/         # Extract, transform, sample and storage
│   ├── config.py     # Central settings
│   ├── pipeline.py   # ETL orchestration
│   └── tests/        # Automated tests

```

The pipeline and dashboard are intentionally separate. API or ingestion failures cannot take down the dashboard because the UI reads only the serving database.

3. Environment setup

Step 1 - Open the project

Start a terminal and change into the repository root.

Step 2 - Create an isolated environment

A virtual environment prevents dependency conflicts with other projects.

Step 3 - Install the package

Editable installation exposes `brand_visibility` while preserving live source edits.

```
cd "/path/to/Brand"
python3 -m venv .venv
source .venv/bin/activate
make install
```

Configuration

Copy `.env.example` to `.env` when using SerpAPI. Never commit the real key.

```
cp .env.example .env
```

```
SERPAPI_KEY=replace_with_your_key
BRAND_DB_PATH=data/database/brand_visibility.db
```

4. Data extraction

The extraction layer sends one Google Shopping request per keyword, validates the HTTP response and maps the external response into the internal product schema. A missing key fails early with a clear message.

Security: The API key is read from `SERPAPI_KEY`. It is never hard-coded in the source or written into exported datasets.

```
python -m brand_visibility.pipeline --api --keywords laptop phone headphones smartwatch
```

Important extraction logic

```
from __future__ import annotations

import os
from typing import Iterable

import pandas as pd

SERPAPI_URL = "https://serpapi.com/search.json"

def extract_google_shopping(
    keywords: Iterable[str], api_key: str | None = None, country: str = "us"
) -> pd.DataFrame:
    """Fetch Google Shopping results from SerpAPI and normalize product records."""
    key = api_key or os.getenv("SERPAPI_KEY")
    if not key:
        raise ValueError("SERPAPI_KEY is required for live extraction")
    try:
        import requests
    except ImportError as exc:
        raise RuntimeError("Install the requirements before using live API extraction") from exc
    records: list[dict] = []
    for keyword in keywords:
        response = requests.get(
            SERPAPI_URL,
            params={"engine": "google_shopping", "q": keyword, "gl": country, "hl": "en", "api_key": key},
            timeout=45,
        )
        response.raise_for_status()
        for item in response.json().get("shopping_results", []):
            records.append(
                {
                    "keyword": keyword,
                    "title": item.get("title"),
```

```

        "price": item.get("extracted_price", item.get("price")),
        "raw_price": item.get("extracted_old_price", item.get("old_price")),
        "rating": item.get("rating"),
        "reviews": item.get("reviews"),
        "platform": item.get("source"),
        "position": item.get("position"),
        "delivery": item.get("delivery", item.get("shipping")),
        "link": item.get("product_link", item.get("link")),
        "thumbnail": item.get("thumbnail"),
        "product_id": item.get("product_id"),
        "source": "serpapi",
    }
)
return pd.DataFrame.from_records(records)

```

5. CSV integration and schema alignment

CSV files from real systems rarely use identical headings. `align\_columns` normalizes case and punctuation, maps common aliases, creates absent fields and returns a canonical column order before concatenation.

Step 1 - Load

Pandas reads the supplied brand_dirty_dataset.csv.

Step 2 - Normalize

Column names become lower snake_case and aliases are mapped.

Step 3 - Complete

Missing canonical fields are created with null values.

Step 4 - Combine

API and CSV rows are appended into one dataframe.

```
python -m brand_visibility.pipeline --csv /path/to/brand_dirty_dataset.csv
```

6. Data cleaning

Issue	Treatment
Price/reviews stored as text	Remove currency/noise characters; safely coerce to numeric
Invalid price or rank	Treat non-positive values as missing
Invalid rating	Accept only values from 0 to 5
Missing numeric values	Keyword median, then dataset median
Missing delivery	Label Unknown
Noisy product titles	Remove unwanted symbols and collapse whitespace

Issue	Treatment
Platform casing	Normalize to title case
Extreme price	Cap at the 99th percentile and record the threshold
Duplicates	Remove repeated keyword + title + platform + price records

Business rule: Products without ratings or reviews are retained because their price, platform and search visibility still contribute useful market evidence.

7. Feature engineering

Feature	Definition
brand	Known brand found in title; otherwise a conservative first-token fallback
visibility_score	100 / position, so rank 1 receives 100
discount_pct	$(\text{raw_price} - \text{price}) / \text{raw_price} \times 100$, clipped to 0-100
price_range	Under \$50, \$50-\$149, \$150-\$499, \$500-\$999, or \$1,000+
rating_range	Under 3, 3-3.9, 4-4.4, or 4.5+
top_10	1 when position ≤ 10 ; otherwise 0

8. SQLite serving layer

The cleaned dataframe replaces the products table in one controlled operation. A composite index accelerates dashboard filters. All user filter values are passed as SQL parameters, which avoids SQL injection and quoting bugs.

```
make pipeline
```

Generated outputs:

- data/processed/cleaned_products.csv
- data/database/brand_visibility.db
- reports/cleaning_report.json

9. Exploratory data analysis

The analysis module returns one named result for every question in the brief. Results include grouped summaries, distributions, top products, cross-tabs and correlations.

Group	Questions covered
General market	Products by keyword; price/rating/review distributions
Brand	Frequency, visibility, ranking, top-10 presence and ratings
Pricing	Brand distributions, ranges, price/rank relationship and maxima
Discounts	Discount adoption, ranking, brand/platform averages and rating relationship
Platforms	Volume, rating, price, ranking and brand mix
Visibility	Ranking distribution, visibility leaders and factor relationships

```
make eda
```

Interpretation: Correlation indicates association, not causation. Search rank may also be affected by relevance, inventory, advertising and personalization factors that are not present in the dataset.

10. Dashboard operation

```
make dashboard
```

Open <http://localhost:8501>. Sidebar selections are translated into parameterized SQLite queries and refresh all tabs.

Tab	Purpose
Overview	Core KPIs, price distribution, keyword volume, platform share and insights
Brand Insights	Brand volume, ratings, visibility and top-10 presence
Pricing Analysis	Price ranges, rank/rating relationships and discounts
Platform Analysis	Platform volume, price, rating, position and brand distribution

Tab	Purpose
Visibility & Ranking	Ranking distribution, correlations and visibility leaders
Product Explorer	Search, sortable top-10 table, charts and CSV download

11. Testing and quality checks

```
make test
make lint
```

The current transformation test verifies numeric cleaning, duplicate removal, brand extraction, visibility calculation and discount calculation. For production, extend tests for empty datasets, API errors, schema drift and boundary values.

- Pipeline exits successfully and records the cleaning diagnostics.
- Clean numeric fields contain no missing values.
- SQLite row count matches the processed CSV.
- All 30 EDA keys are present.
- Dashboard filters return valid subsets and empty selections are handled.
- Secrets are absent from source control.

12. Docker and production operation

```
docker build -t brand-visibility .
docker run --rm -p 8501:8501 brand-visibility
```

The image installs the project package, exposes port 8501 and uses Streamlit's health endpoint. In a larger deployment, mount durable data storage, inject secrets through the platform's secret manager and run the pipeline as a scheduled job separate from the dashboard container.

Production recommendation: Use managed PostgreSQL when multiple dashboard replicas or concurrent writers are required. SQLite is appropriate for a single-instance learning project and small read-heavy deployments.

13. Troubleshooting

Symptom	Resolution
No module named brand_visibility	Activate the environment and run `make install` from the root
SERPAPI_KEY is required	Copy .env.example to .env and add a valid key, or omit --api
Dashboard has no records	Run `make pipeline`; check sidebar filters and database path
Unexpected CSV columns	Add a source alias in ALIASES or update the canonical schema
Charts are empty after filtering	Widen position/rating/brand selections
Port 8501 already in use	Run Streamlit with `--server.port 8502`

Appendix A. Complete source code

The following listings reproduce every authored code and configuration file required to build, run, test and package the project. Generated CSV, SQLite and report outputs are intentionally excluded.

A.1 pyproject.toml

```
[build-system]
requires = ["setuptools>=69", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "brand-visibility-intelligence"
version = "1.0.0"
description = "Production-style e-commerce brand visibility analytics pipeline and dashboard"
readme = "README.md"
requires-python = ">=3.10"
dependencies = ["streamlit>=1.38", "pandas>=2.1", "numpy>=1.26", "plotly>=5.22", "requests>=2.31", "python-dotenv>=1.0"]

[project.optional-dependencies]
dev = ["pytest>=8.0", "ruff>=0.6"]

[tool.setuptools.packages.find]
where = ["src"]

[tool.pytest.ini_options]
pythonpath = ["src"]
testpaths = ["tests"]

[tool.ruff]
line-length = 120
target-version = "py310"
```

A.2 requirements.txt

```
streamlit>=1.38  
pandas>=2.1  
numpy>=1.26  
plotly>=5.22  
requests>=2.31  
python-dotenv>=1.0  
pytest>=8.0
```

A.3 .env.example

```
SERPAPI_KEY=replace_with_your_key  
BRAND_DB_PATH=data/database/brand_visibility.db
```

A.4 .gitignore

```
.env  
.venv/  
__pycache__/  
*.pyc  
data/raw/.gitkeep  
data/interim/*  
!data/interim/.gitkeep  
data/processed/*.csv  
data/database/*.db  
.pytest_cache/  
.ruff_cache/  
dist/  
build/  
*.egg-info/
```

A.5 Makefile

```
.PHONY: install pipeline dashboard eda test lint clean

install:
    python -m pip install -e ".[dev]"
pipeline:
    python -m brand_visibility.pipeline
dashboard:
    streamlit run src/brand_visibility/dashboard/app.py
eda:
    python scripts/export_eda.py
test:
    pytest -q
lint:
    ruff check src tests scripts
clean:
    find . -type d -name __pycache__ -prune -exec rm -rf {} +
    find . -type f -name '*.pyc' -delete
```

A.6 Dockerfile

```
FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 PYTHONPATH=/app/src
WORKDIR /app
COPY pyproject.toml README.md ./
COPY src ./src
RUN pip install --no-cache-dir .
COPY data ./data
COPY reports ./reports
EXPOSE 8501
HEALTHCHECK CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8501/_stcore/health')"
CMD ["streamlit", "run", "src/brand_visibility/dashboard/app.py", "--server.address=0.0.0.0"]
```

A.7 .dockerignore

```
.git  
.venv  
__pycache__  
*.pyc  
.env  
tests  
docs  
data/raw/*  
data/interim/*
```

A.8 .streamlit/config.toml

```
[theme]
primaryColor = "#6C63FF"
backgroundColor = "#F5F7FB"
secondaryBackgroundColor = "#FFFFFF"
textColor = "#172 033"
font = "sans serif"
[server]
headless = true
enableCORS = false
```

A.9 config/logging.ini

```
[loggers]
keys=root,brand_visibility
[handlers]
keys=console
[formatters]
keys=standard
[logger_root]
level=INFO
handlers=console
[logger_brand_visibility]
level=INFO
handlers=console
qualname=brand_visibility
propagate=0
[handler_console]
class=StreamHandler
level=INFO
formatter=standard
args=(sys.stdout,)
[formatter_standard]
format=%(asctime)s | %(levelname)s | %(name)s | %(message)s
```

A.10 src/brand_visibility/__init__.py

```
"""Brand Visibility Intelligence application package."""  
  
__version__ = "1.0.0"
```

A.11 src/brand_visibility/config.py

```
from __future__ import annotations

import os
from dataclasses import dataclass
from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parents[2]

@dataclass(frozen=True)
class Settings:
    """Centralized filesystem and environment configuration."""

    project_root: Path = PROJECT_ROOT
    raw_data_dir: Path = PROJECT_ROOT / "data" / "raw"
    interim_data_dir: Path = PROJECT_ROOT / "data" / "interim"
    processed_data_dir: Path = PROJECT_ROOT / "data" / "processed"
    database_dir: Path = PROJECT_ROOT / "data" / "database"
    reports_dir: Path = PROJECT_ROOT / "reports"

    @property
    def database_path(self) -> Path:
        configured = os.getenv("BRAND_DB_PATH")
        return Path(configured).expanduser() if configured else self.database_dir / "brand_visibility.db"

settings = Settings()
```

A.12 src/brand_visibility/pipeline.py

```

from __future__ import annotations

import argparse
import json
from pathlib import Path

import pandas as pd

try:
    from dotenv import load_dotenv
except ImportError: # Optional convenience; environment variables still work without it.
    def load_dotenv() -> bool:
        return False

from brand_visibility.config import settings
from brand_visibility.data.database import save_products
from brand_visibility.data.extract import extract_google_shopping
from brand_visibility.data.sample_data import generate_sample_data
from brand_visibility.data.transform import clean_and_engineer

def run_pipeline(csv_path: str | None, keywords: list[str], use_api: bool, db_path: str) -> tuple[pd.DataFrame, dict]:
    frames: list[pd.DataFrame] = []
    if csv_path:
        frames.append(pd.read_csv(csv_path))
    if use_api:
        api_df = extract_google_shopping(keywords)
        settings.raw_data_dir.mkdir(parents=True, exist_ok=True)
        api_df.to_csv(settings.raw_data_dir / "api_products.csv", index=False)
        frames.append(api_df)
    if not frames:
        frames.append(generate_sample_data())
    clean, report = clean_and_engineer(frames)
    settings.processed_data_dir.mkdir(parents=True, exist_ok=True)
    clean.to_csv(settings.processed_data_dir / "cleaned_products.csv", index=False)
    save_products(clean, db_path)
    settings.reports_dir.mkdir(parents=True, exist_ok=True)
    (settings.reports_dir / "cleaning_report.json").write_text(json.dumps(report, indent=2))
    return clean, report

if __name__ == "__main__":
    load_dotenv()
    parser = argparse.ArgumentParser(description="Build the Brand Visibility Intelligence dataset")
    parser.add_argument("--csv", help="Path to brand_dirty_dataset.csv")
    parser.add_argument("--api", action="store_true", help="Also fetch live SerpAPI results")
    parser.add_argument("--keywords", nargs="+", default=["laptop", "phone", "headphones", "smartwatch"])
    parser.add_argument("--db", default=str(settings.database_path))
    args = parser.parse_args()
    result, diagnostics = run_pipeline(args.csv, args.keywords, args.api, args.db)
    print(f"Saved {len(result):,} cleaned rows to {args.db}")
    print(json.dumps(diagnostics, indent=2))

```

A.13 src/brand_visibility/data/extract.py

```

from __future__ import annotations

import os
from typing import Iterable

import pandas as pd

SERPAPI_URL = "https://serpapi.com/search.json"

def extract_google_shopping(
    keywords: Iterable[str], api_key: str | None = None, country: str = "us"
) -> pd.DataFrame:
    """Fetch Google Shopping results from SerpAPI and normalize product records."""
    key = api_key or os.getenv("SERPAPI_KEY")
    if not key:
        raise ValueError("SERPAPI_KEY is required for live extraction")
    try:
        import requests
    except ImportError as exc:
        raise RuntimeError("Install the requirements before using live API extraction") from exc
    records: list[dict] = []
    for keyword in keywords:
        response = requests.get(
            SERPAPI_URL,
            params={"engine": "google_shopping", "q": keyword, "gl": country, "hl": "en", "api_key": key},
            timeout=45,
        )
        response.raise_for_status()
        for item in response.json().get("shopping_results", []):
            records.append(
                {
                    "keyword": keyword,
                    "title": item.get("title"),
                    "price": item.get("extracted_price", item.get("price")),
                    "raw_price": item.get("extracted_old_price", item.get("old_price")),
                    "rating": item.get("rating"),
                    "reviews": item.get("reviews"),
                    "platform": item.get("source"),
                    "position": item.get("position"),
                    "delivery": item.get("delivery", item.get("shipping")),
                    "link": item.get("product_link", item.get("link")),
                    "thumbnail": item.get("thumbnail"),
                    "product_id": item.get("product_id"),
                    "source": "serpapi",
                }
            )
    return pd.DataFrame.from_records(records)

```

A.14 src/brand_visibility/data/sample_data.py

```

from __future__ import annotations

import random
from pathlib import Path

import pandas as pd

BRANDS = {
    "laptop": ["Apple", "Dell", "HP", "Lenovo", "Asus", "Acer"],
    "phone": ["Apple", "Samsung", "Google", "OnePlus", "Motorola", "Xiaomi"],
    "headphones": ["Sony", "Bose", "JBL", "Apple", "Sennheiser", "Beats"],
    "smartwatch": ["Apple", "Samsung", "Garmin", "Fitbit", "Amazfit", "Google"],
}
PLATFORMS = ["Amazon", "Walmart", "Best Buy", "Target", "eBay"]

def generate_sample_data(rows_per_keyword: int = 45, seed: int = 42) -> pd.DataFrame:
    """Create deterministic, realistic demo data when source files are unavailable."""
    rng = random.Random(seed)
    rows: list[dict] = []
    for keyword, brands in BRANDS.items():
        for position in range(1, rows_per_keyword + 1):
            brand = rng.choice(brands)
            platform = rng.choices(PLATFORMS, weights=[36, 23, 18, 12, 11])[0]
            base = {"laptop": 850, "phone": 650, "headphones": 180, "smartwatch": 260}[keyword]
            raw_price = round(max(25, rng.gauss(base, base * 0.38)), 2)
            discount = rng.choice([0, 0, 0, 5, 10, 12, 15, 20, 25, 30])
            price = round(raw_price * (1 - discount / 100), 2)
            rating = round(min(5, max(2.8, rng.gauss(4.25, 0.38))), 1)
            reviews = max(0, int(rng.lognormvariate(6.0, 1.35)))
            model = rng.choice(["Pro", "Plus", "Air", "Ultra", "Max", "Essential"])
            rows.append(
                {
                    "keyword": keyword,
                    "title": f"{brand} {model} {keyword.title()} {rng.randint(10, 99)}",
                    "price": price,
                    "raw_price": raw_price,
                    "rating": rating,
                    "reviews": reviews,
                    "platform": platform,
                    "position": position,
                    "delivery": rng.choice(["Free Delivery", "Prime", "Paid", "Unknown"]),
                    "link": f"https://example.com/{keyword}/{position}",
                    "thumbnail": "",
                    "source": "demo",
                }
            )
    return pd.DataFrame(rows)

def write_sample_csv(path: str | Path) -> Path:
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    generate_sample_data().to_csv(path, index=False)
    return path

```

A.15 src/brand_visibility/data/transform.py

```

from __future__ import annotations

import re
from typing import Iterable

import numpy as np
import pandas as pd

BASE_COLUMNS = [
    "keyword", "title", "price", "raw_price", "rating", "reviews", "platform",
    "position", "delivery", "link", "thumbnail", "product_id", "source",
]
KNOWN_BRANDS = [
    "Apple", "Samsung", "Google", "Dell", "HP", "Lenovo", "Asus", "Acer",
    "Sony", "Bose", "JBL", "Sennheiser", "Beats", "OnePlus", "Motorola",
    "Xiaomi", "Garmin", "Fitbit", "Amazfit", "Microsoft", "LG", "Logitech",
]
ALIASES = {
    "search_term": "keyword", "product_title": "title", "name": "title",
    "sale_price": "price", "original_price": "raw_price", "old_price": "raw_price",
    "review_count": "reviews", "source": "platform", "rank": "position",
    "shipping": "delivery", "product_link": "link", "image": "thumbnail",
}

def _numeric(series: pd.Series) -> pd.Series:
    cleaned = series.astype("string").str.replace(r"^[^0-9.\\-]", "", regex=True)
    return pd.to_numeric(cleaned.replace({"": pd.NA, "-": pd.NA}), errors="coerce")

def _clean_title(value: object) -> str:
    text = re.sub(r"[^\\w\\s&+()\\-]", " ", str(value or ""))
    return re.sub(r"\\s+", " ", text).strip()

def _brand_from_title(title: str) -> str:
    lower = title.casefold()
    for brand in KNOWN_BRANDS:
        if re.search(rf"\\b{re.escape(brand.casefold())}\\b", lower):
            return brand
    first = title.split(maxsplit=1)[0] if title else "Unknown"
    return first.title() if first.isalpha() else "Unknown"

def align_columns(frame: pd.DataFrame, source_name: str) -> pd.DataFrame:
    df = frame.copy()
    df.columns = [re.sub(r"^[a-z0-9]+", " ", str(c).strip().lower()).strip(".") for c in df.columns]
    df = df.rename(columns={k: v for k, v in ALIASES.items() if k in df.columns and v not in df.columns})
    if "source" not in df.columns:
        df["source"] = source_name
    for column in BASE_COLUMNS:
        if column not in df.columns:
            df[column] = pd.NA
    return df[BASE_COLUMNS]

def clean_and_engineer(frames: Iterable[pd.DataFrame]) -> tuple[pd.DataFrame, dict]:
    aligned = [align_columns(frame, f"input_{i + 1}") for i, frame in enumerate(frames) if not frame.empty]
    if not aligned:
        raise ValueError("No product rows were supplied")

```

```

df = pd.concat(aligned, ignore_index=True)
report = {"input_rows": len(df), "input_columns": len(df.columns)}
for column in ["price", "raw_price", "rating", "reviews", "position"]:
    df[column] = _numeric(df[column])
df["title"] = df["title"].map(_clean_title)
df["keyword"] = df["keyword"].astype("string").str.strip().str.lower().fillna("unknown")
df["platform"] = df["platform"].astype("string").str.strip().str.title().replace({"<NA>": "Unknown", "": "Unknown"})
df["delivery"] = df["delivery"].astype("string").str.strip().replace({"<NA>": "Unknown", "": "Unknown"})
df.loc[df["price"] <= 0, "price"] = np.nan
df.loc[~df["rating"].between(0, 5), "rating"] = np.nan
df.loc[df["reviews"] < 0, "reviews"] = np.nan
df.loc[df["position"] <= 0, "position"] = np.nan
df["raw_price"] = df["raw_price"].fillna(df["price"])
df.loc[df["raw_price"] < df["price"], "raw_price"] = df["price"]
# Retain unrated products; medians make aggregate analysis usable without inventing extremes.
for column in ["price", "rating", "reviews", "position"]:
    df[column] = df.groupby("keyword")[column].transform(lambda s: s.fillna(s.median()))
    df[column] = df[column].fillna(df[column].median())
price_cap = float(df["price"].quantile(0.99))
df["price"] = df["price"].clip(upper=price_cap)
df["raw_price"] = df["raw_price"].clip(upper=max(price_cap, float(df["raw_price"].quantile(0.99))))
before = len(df)
df = df.drop_duplicates(subset=["keyword", "title", "platform", "price"], keep="first")
df["brand"] = df["title"].map(_brand_from_title)
df["visibility_score"] = (100 / df["position"].clip(lower=1)).round(2)
df["discount_pct"] = np.where(
    df["raw_price"] > 0, ((df["raw_price"] - df["price"]) / df["raw_price"] * 100).clip(0, 100), 0,
).round(2)
bins = [-np.inf, 50, 150, 500, 1000, np.inf]
df["price_range"] = pd.cut(df["price"], bins=bins, labels=["Under $50", "$50-$149", "$150-$499", "$500-$999", "$1,000+"])
df["rating_range"] = pd.cut(df["rating"], bins=[-np.inf, 3, 4, 4.5, np.inf], labels=["Under 3", "3-3.9", "4-4.4", "4.5+"])
df["top_10"] = (df["position"] <= 10).astype(int)
df["reviews"] = df["reviews"].round().astype(int)
df["position"] = df["position"].round().astype(int)
report.update({
    "duplicate_rows_removed": before - len(df), "output_rows": len(df),
    "price_cap_99th_percentile": round(price_cap, 2),
    "remaining_missing_values": int(df[["price", "rating", "reviews", "position"]].isna().sum().sum()),
})
return df.reset_index(drop=True), report

```

A.16 src/brand_visibility/data/database.py

```

from __future__ import annotations

import sqlite3
from pathlib import Path
from typing import Any

import pandas as pd

TABLE = "products"

def connect(db_path: str | Path) -> sqlite3.Connection:
    path = Path(db_path)
    path.parent.mkdir(parents=True, exist_ok=True)
    connection = sqlite3.connect(path, check_same_thread=False)
    connection.row_factory = sqlite3.Row
    return connection

def save_products(df: pd.DataFrame, db_path: str | Path) -> None:
    stored = df.copy()
    for column in stored.select_dtypes(include=["category"]).columns:
        stored[column] = stored[column].astype("string")
    with connect(db_path) as conn:
        stored.to_sql(TABLE, conn, if_exists="replace", index=False)
        conn.executescript(
            "CREATE INDEX IF NOT EXISTS idx_product_filters ON products(keyword, platform, brand, price, rating, position);"
        )

def query_products(db_path: str | Path, filters: dict[str, Any] | None = None, search: str = "") -> pd.DataFrame:
    filters = filters or {}
    clauses, params = ["1=1"], []
    for column in ["brand", "platform", "price_range", "rating_range", "keyword"]:
        values = filters.get(column) or []
        if values:
            clauses.append(f"{column} IN ({','.join('?' for _ in values)})")
            params.extend(values)
    for column in ["price", "rating", "position"]:
        bounds = filters.get(column)
        if bounds:
            clauses.append(f"{column} BETWEEN ? AND ?")
            params.extend([bounds[0], bounds[1]])
    if search.strip():
        clauses.append("LOWER(title) LIKE ?")
        params.append(f"%{search.strip().lower()}%")
    sql = f"SELECT * FROM {TABLE} WHERE {' AND '.join(clauses)}"
    with connect(db_path) as conn:
        return pd.read_sql_query(sql, conn, params=params)

def filter_options(db_path: str | Path) -> dict[str, list]:
    with connect(db_path) as conn:
        return {
            column: [row[0] for row in conn.execute(f"SELECT DISTINCT {column} FROM {TABLE} WHERE {column} IS NOT NULL ORDER BY {column}")]
            for column in ["brand", "platform", "price_range", "rating_range", "keyword"]
        }

```

A.17 src/brand_visibility/analytics/eda.py

```

from __future__ import annotations

import numpy as np
import pandas as pd

def answer_eda_questions(df: pd.DataFrame) -> dict[str, object]:
    """Return reproducible answers/data for every EDA question in the brief."""
    g_brand = df.groupby("brand", observed=True)
    g_platform = df.groupby("platform", observed=True)
    discounted = df[df["discount_pct"] > 0]
    top = df[df["position"] <= 10]
    numeric = df[["price", "rating", "reviews", "position", "discount_pct"]].corr(numeric_only=True)
    return {
        "01_products_per_keyword": df["keyword"].value_counts(),
        "02_overall_average_price": df["price"].mean(),
        "03_price_distribution": df["price"].describe(),
        "04_average_rating": df["rating"].mean(),
        "05_review_distribution": df["reviews"].describe(),
        "06_most_frequent_brand": df["brand"].value_counts().idxmax(),
        "07_brand_highest_visibility": g_brand["visibility_score"].mean().idxmax(),
        "08_average_position_by_brand": g_brand["position"].mean().sort_values(),
        "09_brands_most_in_top_10": top["brand"].value_counts(),
        "10_brand_highest_rating": g_brand["rating"].mean().sort_values(ascending=False),
        "11_price_by_brand": g_brand["price"].describe(),
        "12_price_range_distribution": df["price_range"].value_counts(),
        "13_price_position_correlation": numeric.loc["price", "position"],
        "14_average_price_by_platform": g_platform["price"].mean().sort_values(),
        "15_highest_price_by_keyword": df.loc[df.groupby("keyword")["price"].idxmax(), ["keyword", "title", "price"]],
        "16_percent_discounted": 100 * (df["discount_pct"] > 0).mean(),
        "17_discounted_vs_ranking": df.groupby(df["discount_pct"] > 0)["position"].mean(),
        "18_brand_highest_discount": g_brand["discount_pct"].mean().sort_values(ascending=False),
        "19_platform_highest_discount": g_platform["discount_pct"].mean().sort_values(ascending=False),
        "20_discount_rating_correlation": numeric.loc["discount_pct", "rating"],
        "21_platform_most_products": df["platform"].value_counts(),
        "22_platform_highest_rating": g_platform["rating"].mean().sort_values(ascending=False),
        "23_platform_lowest_price": g_platform["price"].mean().sort_values(),
        "24_average_position_by_platform": g_platform["position"].mean().sort_values(),
        "25_brand_by_platform": pd.crosstab(df["platform"], df["brand"]),
        "26_position_distribution": df["position"].describe(),
        "27_highest_visibility_products": df.nlargest(10, "visibility_score")[["title", "brand", "visibility_score"]],
        "28_rating_position_correlation": numeric.loc["rating", "position"],
        "29_reviews_position_correlation": numeric.loc["reviews", "position"],
        "30_top_rank_factor_comparison": pd.DataFrame({
            "top_10_mean": top[["price", "rating", "reviews"]].mean(),
            "other_mean": df[df["position"] > 10][["price", "rating", "reviews"]].mean(),
        }),
    }

def business_insights(df: pd.DataFrame) -> list[str]:
    if df.empty:
        return ["No products match the current filters."]
    top_brand = df["brand"].value_counts().idxmax()
    best_platform = df.groupby("platform")["rating"].mean().idxmax()
    visible_brand = df.groupby("brand")["visibility_score"].mean().idxmax()
    discount_rank = df.groupby(df["discount_pct"] > 0)["position"].mean()
    rank_note = "better" if len(discount_rank) == 2 and discount_rank.get(True, np.inf) < discount_rank.get(False, np.inf) else "not better"
    return [
        f"{top_brand} has the broadest assortment in the selected market.",
        f"{visible_brand} has the strongest average search visibility.",
    ]

```

```
f"{best_platform} has the highest average customer rating",
f"Discounted products have {rank_note} average rankings than non-discounted products.",
]
```

A.18 src/brand_visibility/dashboard/app.py

```

from __future__ import annotations

from pathlib import Path

import pandas as pd
import plotly.express as px
import streamlit as st

from brand_visibility.analytics.eda import business_insights
from brand_visibility.config import settings
from brand_visibility.data.database import filter_options, query_products
from brand_visibility.pipeline import run_pipeline

st.set_page_config(page_title="Brand Visibility Intelligence", page_icon="💎", layout="wide")
DB_PATH = str(settings.database_path)
COLORS = ["#6C63FF", "#18B6A4", "#FFB547", "#EF6A79", "#4D8BFF", "#A16AE8"]

st.markdown(
    """
    <style>
    .stApp {background: #f5f7fb; color: #172033}
    [data-testid="stSidebar"] {background: #111827}
    [data-testid="stSidebar" ] * {color: #f8fafc}
    [data-testid="stMetric"] {background:white;border:1px solid #e6eaf2;border-radius:14px;padding:16px;box-shadow:0 7px 24px #16223b0b}
    div[data-testid="stPlotlyChart"] {background:white;border:1px solid #e6eaf2;border-radius:14px;padding:8px}
    .block-container {padding-top:1.5rem;max-width:1500px}
    h1,h2,h3 {letter-spacing:-.025em}
    .hero {background:linear-gradient(120deg,#10172a,#3430a0 65%,#635bff);padding:28px 34px;border-radius:18px;color:white;margin-bottom:16px}
    .hero h1 {margin:0;color:white}.hero p{margin:.45rem 0 0;color:#dfe3ff}
    .insight {background:#fff;border-left:4px solid #6C63FF;padding:12px 14px;margin:8px 0;border-radius:8px}
    </style>
    """
    ,
    unsafe_allow_html=True,
)

def ensure_database() -> None:
    if not Path(DB_PATH).exists():
        run_pipeline(None, [], False, DB_PATH)

@st.cache_data(show_spinner=False)
def options(db_mtime: float) -> dict[str, list]:
    return filter_options(DB_PATH)

def card_row(items: list[tuple[str, str]]) -> None:
    for column, (label, value) in zip(st.columns(len(items)), items):
        column.metric(label, value)

def money(value: float) -> str:
    return f"${value:,.2f}" if pd.notna(value) else "-"

def base_layout(fig, height: int = 340):
    fig.update_layout(
        height=height, margin=dict(l=20, r=20, t=55, b=20), paper_bgcolor="white",
        plot_bgcolor="white", colorway=COLORS, legend_title_text="",
    )

```

```

fig.update_xaxes(gridcolor="#eef1f6")
fig.update_yaxes(gridcolor="#eef1f6")
return fig

ensure_database()
opts = options(Path(DB_PATH).stat().st_mtime)

with st.sidebar:
    st.markdown("## ♦ Intelligence Hub")
    st.caption("SQL-connected global filters")
    selected_brand = st.multiselect("Brand", opts["brand"])
    selected_platform = st.multiselect("Platform", opts["platform"])
    selected_price_range = st.multiselect("Price range", opts["price_range"])
    selected_rating_range = st.multiselect("Rating range", opts["rating_range"])
    selected_keyword = st.multiselect("Keyword", opts["keyword"])
    position = st.slider("Position (ranking)", 1, 50, (1, 50))
    st.divider()
    st.caption("Every selection is translated into a parameterized SQLite query.")

filters = {
    "brand": selected_brand, "platform": selected_platform, "price_range": selected_price_range,
    "rating_range": selected_rating_range, "keyword": selected_keyword, "position": position,
}
df = query_products(DB_PATH, filters)

st.markdown(
    "<div class='hero'><h1>Brand Visibility Intelligence</h1><p>Search performance, pricing, customer trust and competitive position in one view.</p></div>",
    unsafe_allow_html=True,
)
if df.empty:
    st.warning("No products match these filters. Widen one or more sidebar selections.")
    st.stop()

overview, brands, pricing, platforms, visibility, explorer = st.tabs(
    ["Overview", "Brand Insights", "Pricing Analysis", "Platform Analysis", "Visibility & Ranking", "Product Explorer"]
)

with overview:
    card_row([
        ("Total products", f"{len(df):,}"), ("Average price", money(df.price.mean())),
        ("Average rating", f"{df.rating.mean():.2f} ★"), ("Total reviews", f"{df.reviews.sum():.0f}"),
        ("Avg visibility", f"{df.visibility_score.mean():.1f}"),
    ])
    c1, c2 = st.columns(2)
    with c1:
        st.plotly_chart(base_layout(px.histogram(df, x="price", nbins=30, title="Price distribution", color_discrete_sequence=COLORS)), use_container_width=True)
        per_keyword = df.keyword.value_counts().rename_axis("keyword").reset_index(name="products")
        st.plotly_chart(base_layout(px.bar(per_keyword, x="keyword", y="products", title="Products per keyword", color="keyword", color_discrete_sequence=COLORS)), use_container_width=True)
    with c2:
        share = df.platform.value_counts().rename_axis("platform").reset_index(name="products")
        st.plotly_chart(base_layout(px.pie(share, names="platform", values="products", hole=.55, title="Platform share", color_discrete_sequence=COLORS)), use_container_width=True)
        reviews = df.assign(engagement=pd.cut(df.reviews, [-1, 100, 1000, 10000, float("inf")], labels=["Low", "Growing", "High", "Viral"]))
        engagement = reviews.engagement.value_counts().rename_axis("engagement").reset_index(name="products")
        st.plotly_chart(base_layout(px.bar(engagement, x="engagement", y="products", title="Review engagement distribution", color="engagement", color_discrete_sequence=COLORS)), use_container_width=True)
    st.subheader("What the data says")
    for insight in business_insights(df):
        st.markdown(f"<div class='insight'>{insight}</div>", unsafe_allow_html=True)

with brands:
    brand_stats = df.groupby("brand", as_index=False).agg(products=("title", "count"), avg_rating=("rating", "mean"), avg_visibility=("visibility_score", "mean"), avg_position=("position", "mean"))
    top_brand = brand_stats.loc[brand_stats.products.idxmax(), "brand"]
    rated_brand = brand_stats.loc[brand_stats.avg_rating.idxmax(), "brand"]
    visible_brand = brand_stats.loc[brand_stats.avg_visibility.idxmax(), "brand"]

```

```

card_row(["Total brands", f"{df.brand.nunique():,}"), ("Top brand", top_brand), ("Highest rated", rated_brand), ("Best visibility", visible_brand)])
c1, c2 = st.columns(2)
c1.plotly_chart(base_layout(px.bar(brand_stats.nlargest(15, "products"), x="brand", y="products", title="Brand vs product count", color="products", color_continuous_scale="Purples")), use_container_width=True)
c2.plotly_chart(base_layout(px.bar(brand_stats.nlargest(15, "avg_rating"), x="brand", y="avg_rating", title="Brand vs average rating", color="avg_rating", color_continuous_scale="Teal")), use_container_width=True)
top10 = df[df.position <= 10].brand.value_counts().head(15).rename_axis("brand").reset_index(name="top_10_products")
st.plotly_chart(base_layout(px.bar(top10, x="top_10_products", y="brand", orientation="h", title="Brands appearing most often in top 10", color="top_10_products", color_continuous_scale="Bluered")), use_container_width=True)

with pricing:
    high = df.loc[df.price.idxmax()]
    low = df.loc[df.price.idxmin()]
    card_row(["Average price", money(df.price.mean())], ("Highest price", money(high.price)), ("Lowest price", money(low.price)), ("Discounted products", f"{(df.discount_pct.gt(0).mean() * 100):.1f}%"))
    c1, c2 = st.columns(2)
    c1.plotly_chart(base_layout(px.histogram(df, x="price", nbins=30, color="price_range", title="Price distribution", color_discrete_sequence=COLORS)), use_container_width=True)
    c2.plotly_chart(base_layout(px.scatter(df, x="price", y="position", color="platform", hover_name="title", title="Price vs ranking", color_discrete_sequence=COLORS)), use_container_width=True)
    c1.plotly_chart(base_layout(px.scatter(df, x="price", y="rating", color="brand", hover_name="title", title="Price vs rating", color_discrete_sequence=COLORS)), use_container_width=True)
    ranges = df.price_range.value_counts().rename_axis("price_range").reset_index(name="products")
    c2.plotly_chart(base_layout(px.bar(ranges, x="price_range", y="products", title="Price range distribution", color="price_range", color_discrete_sequence=COLORS)), use_container_width=True)
    discounts = df.groupby("brand", as_index=False).discount_pct.mean().nlargest(15, "discount_pct")
    platform_discount = df.groupby("platform", as_index=False).discount_pct.mean()
    c1.plotly_chart(base_layout(px.bar(discounts, x="brand", y="discount_pct", title="Average discount by brand", color="discount_pct", color_continuous_scale="Purples")), use_container_width=True)
    c2.plotly_chart(base_layout(px.bar(platform_discount, x="platform", y="discount_pct", title="Average discount by platform", color="platform", color_discrete_sequence=COLORS)), use_container_width=True)

with platforms:
    stats = df.groupby("platform", as_index=False).agg(products=("title", "count"), avg_price=("price", "mean"), avg_rating=("rating", "mean"), avg_position=("position", "mean"))
    card_row(["Total platforms", str(len(stats))], ("Best platform", stats.loc[stats.avg_rating.idxmax(), "platform"]), ("Cheapest platform", stats.loc[stats.avg_price.idxmin(), "platform"]), ("Most products", stats.loc[stats.products.idxmax(), "platform"]))
    c1, c2 = st.columns(2)
    for target, title, slot in [("products", "Platform vs product count", c1), ("avg_rating", "Platform vs average rating", c2), ("avg_price", "Platform vs average price", c1), ("avg_position", "Platform vs average ranking", c2)]:
        slot.plotly_chart(base_layout(px.bar(stats, x="platform", y=target, title=title, color="platform", color_discrete_sequence=COLORS)), use_container_width=True)
    matrix = df.groupby(["platform", "brand"], as_index=False).size()
    st.plotly_chart(base_layout(px.bar(matrix, x="platform", y="size", color="brand", title="Brand distribution per platform", color_discrete_sequence=COLORS)), use_container_width=True)

with visibility:
    best = df.loc[df.position.idxmin(), "title"]
    card_row(["Average position", f"{df.position.mean():.1f}"], ("Best-ranked product", best), ("Avg visibility", f"{df.visibility_score.mean():.1f}"), ("Products in top 10", f"{df.top_10.mean() * 100:.1f}%"))
    c1, c2 = st.columns(2)
    c1.plotly_chart(base_layout(px.histogram(df, x="position", nbins=30, title="Ranking distribution", color_discrete_sequence=COLORS)), use_container_width=True)
    c2.plotly_chart(base_layout(px.scatter(df, x="rating", y="position", color="platform", hover_name="title", title="Rating vs ranking", color_discrete_sequence=COLORS)), use_container_width=True)
    c1.plotly_chart(base_layout(px.scatter(df, x="reviews", y="position", size="visibility_score", color="brand", hover_name="title", title="Reviews vs ranking", color_discrete_sequence=COLORS)), use_container_width=True)
    vis = df.groupby("brand", as_index=False).visibility_score.mean().nlargest(15, "visibility_score")
    c2.plotly_chart(base_layout(px.bar(vis, x="brand", y="visibility_score", title="Visibility score by brand", color="visibility_score", color_continuous_scale="Purples")), use_container_width=True)

with explorer:
    search = st.text_input("Search product title", placeholder="Try Apple, Pro, laptop...")
    products = query_products(DB_PATH, filters, search)
    card_row(["Filtered products", f"{len(products):,}"), ("Average price", money(products.price.mean())], ("Average rating", f"{products.rating.mean():.2f} ★"))
    if not products.empty:
        top_products = products.sort_values(["visibility_score", "rating", "reviews"], ascending=[False, False, False]).head(10)
        display_cols = ["title", "brand", "price", "rating", "reviews", "platform", "position", "discount_pct", "visibility_score", "link"]
        st.subheader("Top 10 products")
        st.dataframe(
            top_products[display_cols], use_container_width=True, hide_index=True,
            column_config={"price": st.column_config.NumberColumn(format="${:.2f}"), "rating": st.column_config.ProgressColumn(min_value=0, max_value=5), "link": st.column_config.LinkColumn("Product link")},
        )
        c1, c2 = st.columns(2)
        c1.plotly_chart(base_layout(px.scatter(products, x="price", y="rating", color="platform", hover_name="title", title="Rating vs price", color_discrete_sequence=COLORS)), use_container_width=True)
        c2.plotly_chart(base_layout(px.scatter(products, x="reviews", y="position", size="visibility_score", color="platform", hover_name="title", title="Reviews vs ranking", color_discrete_sequence=COLORS)), use_container_width=True)
        st.download_button("Download filtered CSV", products.to_csv(index=False), "filtered_products.csv", "text/csv")

```

A.19 scripts/export_eda.py

```
from __future__ import annotations

from pathlib import Path

from brand_visibility.analytics.eda import answer_eda_questions
from brand_visibility.config import settings
from brand_visibility.data.database import query_products

def main() -> None:
    answers = answer_eda_questions(query_products(settings.database_path))
    out = settings.reports_dir / "eda_answers.md"
    out.parent.mkdir(parents=True, exist_ok=True)
    blocks = ["# Brand Visibility - 30 EDA Answers\n"]
    for key, value in answers.items():
        title = key.replace("_", " ").title()
        blocks.append(f"## {title}\n\n```\n{text\n{value}\n```\n")
    out.write_text("\n".join(blocks))
    print(f"Wrote {out}")

if __name__ == "__main__":
    main()
```

A.20 tests/test_transform.py

```
import pandas as pd

from brand_visibility.data.transform import clean_and_engineer

def test_cleaning_and_features():
    dirty = pd.DataFrame({
        "keyword": ["Phone", "Phone", "Phone"],
        "title": ["Apple!!! Phone", "Apple!!! Phone", "Samsung Phone"],
        "price": ["$999.00", "$999.00", "Not Available"],
        "raw_price": ["$1,099", "$1,099", "$800"],
        "rating": ["4.8", "4.8", "many"],
        "reviews": ["1,200", "1,200", "20"],
        "platform": ["amazon", "AMAZON", "Walmart"],
        "position": [1, 1, 2],
    })
    clean, report = clean_and_engineer(dirty)
    assert len(clean) == 2
    assert clean.loc[0, "brand"] == "Apple"
    assert clean.loc[0, "visibility_score"] == 100
    assert clean.loc[0, "discount_pct"] > 0
    assert report["duplicate_rows_removed"] == 1
```

Appendix B. Command reference

Task	Command
Install	<code>python -m pip install -e "[dev]"</code>
Demo pipeline	<code>python -m brand_visibility.pipeline</code>
CSV pipeline	<code>python -m brand_visibility.pipeline --csv /path/file.csv</code>
Live API	<code>python -m brand_visibility.pipeline --api --keywords laptop phone</code>
Dashboard	<code>streamlit run src/brand_visibility/dashboard/app.py</code>
EDA export	<code>python scripts/export_eda.py</code>
Tests	<code>pytest -q</code>
Lint	<code>ruff check src tests scripts</code>
Docker	<code>docker build -t brand-visibility .</code>